Review Analyzer

Project Documentation

AI-Powered Business Review Analysis using RAG & Gemini

Table of Contents

1. Project Overview
2. System Architecture
3. Technology Stack
4. Directory Structure
5. Backend Modules
6. Frontend Modules
7. API Reference
8. Data Flow Pipeline
9. Setup & Installation
10. Key Design Decisions

1. Project Overview

Review Analyzer is a AI-powered application that fetches real Google Maps reviews for any business, stores them in a vector database, and uses Retrieval-Augmented Generation (RAG) with Google's Gemini LLM to produce a structured pros-and-cons summary.

How It Works

Step	Action
1	User enters a business name (e.g. 'Taj Palace Hotel Mumbai')
2	Backend fetches live reviews from Google Maps via SerpApi
3	Reviews are embedded and stored in ChromaDB vector database
4	Semantic search retrieves the most relevant reviews (RAG)
5	Gemini 2.5 Flash generates a structured markdown summary
6	React frontend renders the summary with metrics and raw reviews

Core Capabilities

Live Review Fetching: Pulls real-time Google Maps reviews via the SerpApi service.

Vector Storage: Embeds review text with gemini-embedding-001 and persists in ChromaDB.

RAG Retrieval: Semantic similarity search surfaces the most relevant reviews for analysis.

AI Summarization: Gemini 2.5 Flash produces structured markdown summaries with pros, cons, and ratings.

Scheduled Ingestion: An n8n workflow definition enables automated daily review ingestion.

2. System Architecture

The application follows a layered architecture with a clear separation between the frontend presentation layer and the backend processing pipeline. The backend is further decomposed into agents, tools, and a vector store layer.

Architecture Overview

```
┌──────────────────────────────────────────────────────────┐
│              FRONTEND  (React 19 + Vite 8)                │
│   Search Input → Loading UI → Results (Metrics + MD)      │
└──────────────────────────────────────────────────────────┘
                        │
                        │  HTTP POST /api/analyze
                        ▼
┌──────────────────────────────────────────────────────────┐
│              BACKEND  (FastAPI + Uvicorn)                 │
│                                                           │
│   Orchestrator Agent                                      │
│     ├─ Step 1: Review Fetcher  ──→  SerpApi               │
│     ├─ Step 2: ChromaDB Store  ──→  Embed + Persist       │
│     ├─ Step 3: RAG Retriever   ──→  Semantic Search       │
│     └─ Step 4: Summarizer      ──→  Gemini 2.5 Flash      │
└──────────────────────────────────────────────────────────┘


External Services:
  • Google Gemini API  — LLM generation + text embeddings
  • SerpApi            — Google Maps review scraping
  • ChromaDB           — Local vector database (on disk)
```

Request Lifecycle

1. User submits a business name via the React frontend.

2. Frontend sends POST /api/analyze to the FastAPI backend.

3. Orchestrator calls Review Fetcher → SerpApi fetches live Google Maps reviews.

4. Fetched reviews are embedded via gemini-embedding-001 and stored in ChromaDB.

5. RAG Retriever performs semantic search on ChromaDB (top-k = 20 results).

6. Retrieved reviews are sent to Gemini 2.5 Flash with a structured system prompt.

7. Gemini returns a markdown summary (pros, cons, ratings, notable mentions).

8. FastAPI packages everything into a JSON response and sends it to the frontend.

9. React renders the metrics grid, markdown summary, and expandable raw reviews.

3. Technology Stack

Backend

Technology	Purpose	Version
Python	Core language	3.10+
FastAPI	REST API framework	≥ 0.111.0
Uvicorn	ASGI server	≥ 0.30.0
google-genai	Gemini LLM + embeddings SDK	≥ 1.0.0
google-adk	Agent Development Kit	≥ 1.0.0
ChromaDB	Vector database	≥ 0.5.0
google-search-results	SerpApi (Google Maps reviews)	≥ 2.4.2
python-dotenv	Environment variable management	≥ 1.0.0
Pydantic	Request/response validation	≥ 2.7.0

Frontend

Technology	Purpose	Version
React	UI library	^19.2.5
Vite	Build tool & dev server	^8.0.10
react-markdown	Render AI-generated markdown	^10.1.0
Vanilla CSS	Styling	—

External Services

Service	Purpose
Google Gemini API	LLM text generation (gemini-2.5-flash) and embedding (gemini-embedding-001)
SerpApi	Fetching live Google Maps reviews (100 free searches/month)

4. Directory Structure

```
Review-Analyzer/
├── backend/
│   ├── agents/
│   │   ├── orchestrator.py      # Main pipeline coordinator
│   │   └── summarizer.py        # Summarizer system prompt
│   ├── tools/
│   │   ├── review_fetcher.py    # SerpApi integration
│   │   └── rag_retriever.py     # Semantic search tool
│   ├── vectorstore/
│   │   ├── chroma_store.py      # ChromaDB CRUD operations
│   │   └── embeddings.py        # Gemini embedding generation
│   ├── data/
│   │   └── sample_reviews.json  # 26 sample reviews
│   ├── n8n/
│   │   └── review_ingestion_workflow.json
│   ├── main.py                  # FastAPI entry point
│   ├── requirements.txt
│   └── .env / .env.example
├── frontend/
    ├── src/
    │   ├── App.jsx              # Main React component
    │   ├── index.css            # Global styles
    │   └── main.jsx             # React entry point
    ├── index.html
    ├── package.json
    └── vite.config.js
```

5. Backend Modules

5.1 main.py — API Entry Point

Initializes the FastAPI application with CORS middleware (all origins allowed for development). Defines two endpoints: POST /api/analyze for business analysis and GET /api/health for health checks. The analyze endpoint accepts a business_name (required) and location (optional), delegates to the orchestrator's analyze_business() function, and returns the result. Business logic errors return HTTP 400; unexpected exceptions return HTTP 500.

5.2 orchestrator.py — Pipeline Coordinator

The central module containing the analyze_business() function that orchestrates the four-step pipeline: (1) fetch reviews via SerpApi, (2) retrieve relevant reviews via semantic search, (3) summarize using Gemini 2.5 Flash, and (4) return structured results.

Key details: Creates a genai.Client using GOOGLE_API_KEY. The RAG query is constructed as 'Complete analysis of {business} covering service, quality, value, ambiance, and overall experience' to ensure broad topic coverage. Retrieves up to 20 reviews (top_k=20). Uses temperature=0.7 for balanced creativity. The summarizer system instruction is passed as the system_instruction parameter.

5.3 summarizer.py — Summarizer System Prompt

Contains SUMMARIZER_INSTRUCTION, a detailed system prompt that instructs Gemini to produce output in a strict markdown format with five sections: Overall Summary (with sentiment, average rating, review count), Top 5 Pros, Top 5 Cons, Notable Mentions, and Rating Distribution. Rules enforce evidence-based analysis, pattern identification, and balanced coverage.

5.4 review_fetcher.py — SerpApi Integration

Fetches live Google Maps reviews using a two-step process: (1) search the google_maps engine to get the place's data_id, and (2) query google_maps_reviews with that data_id to retrieve reviews. Each review is normalized into a dict with business_name, author, rating, date, and text fields. After fetching, all reviews are automatically embedded and stored in ChromaDB via add_reviews().

5.5 rag_retriever.py — Semantic Search Tool

Performs semantic similarity search on ChromaDB. Calls search_reviews() with the query, optional business name filter, and top_k limit. Formats results into a readable context string for the LLM, including review number, rating, author, date, and full text.

5.6 chroma_store.py — Vector Database Layer

Manages all ChromaDB interactions using persistent storage at backend/chroma_db/. Key functions:

Function	Description
add_reviews()	Embeds and stores reviews, skipping duplicates via MD5 hash IDs
search_reviews()	Semantic similarity search with optional business name filtering
get_all_reviews_for_business()	Retrieves all stored reviews for a specific business
list_businesses()	Returns sorted list of all unique business names in the store

Deduplication: Each review gets an ID via MD5(business_name + author + first 100 chars of text). Before insertion, existing IDs are checked and duplicates are skipped.

5.7 embeddings.py — Embedding Generation

Uses the gemini-embedding-001 model via the google-genai SDK. Provides two functions: get_embeddings(texts) for batch embedding (processes in batches of 100 per API limit) and get_query_embedding(query) for single query embedding. Both return float vectors.

5.8 n8n Workflow — Automated Ingestion

A pre-configured n8n workflow JSON for automated daily review ingestion. The workflow consists of five nodes: Daily Schedule (fires every 24h) → Fetch Reviews (SerpApi HTTP request) → Transform Reviews

(JavaScript normalization) → Send to ChromaDB (HTTP POST to backend) → Log Completion. This file can be imported into any n8n instance.

6. Frontend Modules

6.1 App.jsx — Main Component

A single-component React application handling the complete user interaction flow. Manages six state variables: query (search input), loading (boolean), loadingStep (animation index), result (API response), error (message), and reviewsExpanded (toggle).

Loading Animation

During API calls, a simulated multi-step progress indicator is displayed with timed steps: searching (0s), fetching (1s), storing in vector DB (2.5s), running retrieval (4s), AI analyzing (5.5s), and generating summary (7s).

UI Sections

Header: Application title and subtitle

Search Bar: Text input with Enter-key support and Analyze button

Metrics Grid: Four cards showing review count, average rating, sentiment, and source

Summary: AI-generated markdown rendered via react-markdown

Raw Reviews: Expandable accordion listing all fetched reviews with ratings and metadata

6.2 Styling (index.css)

Clean, minimalist CSS with white background, sans-serif font, and max-width 800px centered layout. The analyze button features a gradient (#4F46E5 → #6366F1) with hover/active transitions and box-shadow effects. Metric cards and summary containers use simple 1px borders.

7. API Reference

POST /api/analyze

Analyze a business by fetching reviews, performing RAG, and generating an AI summary.

Request Body

```
{
  "business_name": "Taj Palace Hotel Mumbai",   // required
  "location": "Mumbai, India"                   // optional
}
```

Success Response (200)

```
{
  "status": "success",
  "business_name": "Taj Palace Hotel Mumbai",
  "summary": "## Overall Summary\n...",
  "review_count": 10,
  "source": "serpapi",
  "reviews": [ { "author": "...", "rating": 5, ... } ]
}
```

Error Responses

Code	Condition	Body
400	No reviews found	{"detail": "No reviews found..."}
500	Unexpected server error	{"detail": "<error message>"}

8. Data Flow Pipeline

Step 1 — User Input: User types a business name in the React search bar and clicks Analyze. The frontend sends a POST request to http://localhost:8000/api/analyze.

Step 2 — Review Fetching: The review_fetcher.py module queries SerpApi's google_maps engine to resolve the business name to a data_id, then fetches reviews from google_maps_reviews. Each review is normalized into a standardized dict (business_name, author, rating, date, text).

Step 3 — Embedding & Storage: The chroma_store.py module generates MD5-based IDs for deduplication, embeds review texts via gemini-embedding-001 (in batches of 100), and stores embeddings with metadata in the 'business_reviews' ChromaDB collection. Data persists to backend/chroma_db/.

Step 4 — RAG Retrieval: The rag_retriever.py module embeds the analysis query, performs cosine similarity search in ChromaDB with a business_name filter, and retrieves the top 20 most relevant reviews. Results are formatted into a human-readable context string.

Step 5 — AI Summarization: The orchestrator builds a prompt with the retrieved reviews and sends it to Gemini 2.5 Flash with the summarizer system instruction. Temperature is set to 0.7. The model returns a structured markdown summary with pros, cons, ratings, and notable mentions.

Step 6 — Response & Rendering: FastAPI packages the summary, metrics, and raw reviews into a JSON response. The React frontend renders four metric cards (review count, avg rating, sentiment, source), the markdown summary via react-markdown, and an expandable raw reviews accordion.

9. Setup & Installation

Prerequisites

- Python 3.10 or higher
- Node.js 18 or higher
- npm 9 or higher

Environment Variables

Create a .env file in the backend/ directory with the following keys:

Variable	Required	Description
GOOGLE_API_KEY	Yes	Google Gemini API key (from aistudio.google.com/apikey)
SERPAPI_KEY	Yes	SerpApi key (from serpapi.com, 100 free searches/month)

Backend Setup

```
cd backend
python -m venv venv
venv\Scripts\activate          # Windows
# source venv/bin/activate     # macOS/Linux
pip install -r requirements.txt
cp .env.example .env
# Edit .env and add your API keys
```

Frontend Setup

```
cd frontend
npm install
```

Running the Application

Start the backend (Terminal 1):

```
cd backend
uvicorn main:app --reload --port 8000
```

Start the frontend (Terminal 2):

```
cd frontend
npm run dev
```

The backend is available at http://localhost:8000 . The UI is available at http://localhost:5173.